

Agentic AI Banking Demo - AWS Decision Rationale and Defense Guide

Architecture decisions, trade-offs, security boundaries, and interview defense

Amazon Bedrock	Aurora PostgreSQL + pgvector	IAM / SSM / Secrets
Generation + embeddings	Facts + vector retrieval + audit	Workload identity + secure access

Design principle: AI can investigate, assemble evidence, interpret authorized policy, and recommend the next action - while the human remains accountable for the consequential decision.

1. Architecture decision summary

The AWS edition intentionally changes cloud services without changing the governing control pattern. Oracle AI Database and local Ollama are replaced with PostgreSQL/pgvector and Amazon Bedrock, while authorization, context minimization, human approval, and audit remain external to the foundation model.

Defend the design by starting with the control flow, not with the model name: authorize first -> retrieve only permitted evidence -> generate -> stop for approval -> audit.

2. Why Amazon Bedrock instead of a locally hosted LLM

- Removes the need to install and operate a local inference server on the demo node.
- Uses AWS workload identity rather than application-embedded API keys.
- Makes model selection a configuration decision while governance stays in application code and data state.
- Creates a clearer path to enterprise network, logging, quota, and model-governance controls.

Trade-off: Bedrock introduces a managed-service dependency and requires Region/model availability planning. The architecture therefore keeps business authorization and workflow state outside the model.

3. Why Titan Text Embeddings V2

The project uses Titan Text Embeddings V2 as the AWS-native embedding service. The default project configuration uses 1024 dimensions, matching the pgvector column definition.

- Keep embedding dimension configurable.
- Re-embed the corpus when switching embedding models or dimensions.
- Do not treat semantic similarity as an authorization decision.

4. Why Aurora PostgreSQL / RDS PostgreSQL + pgvector

- Preserves ordinary SQL for deterministic transaction and account evidence.
- Stores vectorized policy alongside classification and allowed_role metadata.
- Stores action state and audit events in the same compact proof-of-concept data platform.
- Uses the pgvector cosine distance operator for semantic policy ranking.

This is a proof-of-concept simplification, not a universal claim that all production vector search belongs in the operational database. At larger scale, a separate retrieval platform may be appropriate if the control boundary remains intact.

5. Why role filtering happens before vector ranking

Vector search answers relevance, not authorization. Therefore, the SQL predicate first limits eligible rows to policy allowed for the employee role, and only those rows participate in similarity ranking.

This prevents a prompt injection or model behavior change from retrieving a policy row the caller was never entitled to access.

6. Why fraud fields are selected deterministically

For CUSTOMER_SERVICE, the data-access function selects only safe fraud fields. `fraud\_score` and `internal\_notes` are never returned. FRAUD_ANALYST can use a different deterministic query shape.

The model is not asked to decide which fields to hide. It never receives the restricted fields in the first place.

7. Why human approval is a database state transition

- The AI creates a PENDING record instead of executing the action.
- The human review function changes state to APPROVED or REJECTED.
- The human decision produces a separate audit event.
- The approval mechanism can later move behind a web UI or action service without rewriting the AI control thesis.

8. Why IAM roles and Secrets Manager

AWS access should come from workload identity. Database passwords belong in a secret store rather than source code or Git history. The demonstration therefore supports EC2 IAM roles and Secrets Manager.

A local `.env` remains convenient for non-secret configuration and workstation testing, but long-lived AWS access keys should not be added to it.

9. Why Session Manager for the demo node

Session Manager provides an administrative path to EC2 without requiring routine inbound SSH exposure. It also aligns the live demo with an enterprise story about centralized access control and logging.

This does not eliminate the need for network egress or appropriate VPC endpoints. It changes the administrative access pattern.

10. Why not use Bedrock Agents in the first version

The proof of concept deliberately keeps tool orchestration in readable Python so an interviewer or architecture reviewer can see exactly where authorization, retrieval, prompt assembly, action creation, and human review occur.

A managed agent framework can be evaluated later, but the control boundaries should remain explicit and independently testable. The goal is to demonstrate enterprise architecture judgment rather than maximize managed-service count.

11. What this demo proves - and what it does not

Proves	Does not claim
Role-filtered RAG	Production banking certification
Human-in-the-loop consequential action	Autonomous financial decisioning
Auditable AI workflow stages	Complete enterprise observability stack

Proves	Does not claim
AWS-managed model invocation	Zero-risk model output
Portable governance outside the model	A single best architecture for every workload

12. Interview defense - concise answers

Why is this agentic rather than a chatbot?

It pursues a goal, invokes multiple tools, gathers evidence, retrieves policy, reasons over a bounded context, creates workflow state, stops at an approval boundary, and records the outcome.

What is the most important security control?

The model is downstream of deterministic authorization. Restricted data and policy are removed before context assembly.

What would you change first for production?

Enterprise identity, private networking, secrets/KMS, centralized observability, formal evaluation, resilient deployment, and authenticated action APIs with segregation of duties.

AWS reference sources

- AWS: Nova Lite model card - <https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-lite.html>
- AWS: Titan Text Embeddings models - <https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html>
- AWS: Aurora PostgreSQL vector database - <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraPostgreSQL.VectorDB.html>
- AWS: Systems Manager Session Manager - <https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html>